ASSIGNMENT 2 – CO2

C++ Functions and Object-Oriented Programming

Individual Assignment Report

Student Details	Information
Student Name	Jai Ganesh Rao N
Register Number	192524073
Course / Subject	DSA0114- OBJECT ORIENTED PROGRAMMING WITH C++
Assignment	Assignment 2 – CO2
Submission Date	12 November 2026
GitHub Repository	https://github.com/jaiganeshrao/DSA0114_192524073/tree/main/Assignment
Compiler and Version	C++

Declaration

I declare that this report is prepared for the individual Assignment 2 – CO2. The programs, predictions, explanations and evaluation should be checked against my own compilation and execution before submission. All execution screenshots, compiler messages and measured timings must be added from my own system.

Assignment Overview

This assignment applies C++ functions and object-oriented programming concepts: call by value, call by reference, pointers, return by reference, default arguments, function overloading, classes, access specifiers, arrays inside classes, static members, arrays of objects, friend functions and inline functions.

The assignment assesses evaluation (K5). Therefore, each task includes not only an implementation but also reasoning about correctness, limitations, alternative designs, time complexity and space complexity.

Submission and Execution Checklist

- Create a GitHub repository named DSA01-[RegisterNumber]-Assignment2.
- Create separate folders Task01 through Task07.
- Save every C++ source file, required input file and output screenshot in its task folder.
- Add a README.md containing student details, compiler/version, compile commands, run instructions and a short result for every task.
- Commit work progressively, with at least one meaningful commit per task.
- Paste the working GitHub repository link on this report's cover page.
- Replace all evidence placeholders in this report with genuine screenshots, compiler warnings and timing measurements.

NOTE: This report is a structured working draft based on the supplied assignment sheet. Do not claim that a program was executed or that timings were measured until you have performed those steps yourself.

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

1.1 Problem Understanding

The hostel office applies a percentage concession to a student's fee. The key issue is whether a function changes only its local parameter or changes the original variable in main().

1.2 Three Ways of Passing the Fee

Method	Parameter form	Effect on caller	Reason
Call by value	long long fee	No change	A separate copy is modified.
Call by reference	long long& fee	Caller changes	The parameter is an alias for the caller's variable.
Call by pointer	long long* fee	Caller changes when dereferenced	The pointer contains the caller variable's address.

1.3 Prediction for the Given Program

```
#include <iostream>
void change(int a, int& b, int* c)
{
   a += 10;
   b += 10;
```

```
    *c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10)
{
    std::cout << x + y + z << ' ';
}
int main()
{
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';
    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\n';
    show(1);
    show(1, 2);
    show(1, 2, 3);
    std::cout << '\n';
}
```

Predicted output:

```
1 12 13
100 13
16 13 6
```

Explanation: a is passed by value, so its local copy becomes 11 but the caller's a remains 1. b is passed by reference, so b becomes 12. c is passed by pointer and *c updates the original c to 13. In bigger(a, c), c is larger (13), so the returned reference refers to c; assigning 100 changes c to 100. Therefore, the second output line should be 1 100, not 100 13.

Corrected prediction after tracing the reference assignment:

```
1 12 13
1 100
16 13 6
```

The show() calls use these arguments: show(1) uses y=5 and z=10, giving 16; show(1,2) uses z=10, giving 13; show(1,2,3) uses no defaults, giving 6.

1.4 Complete the Code – Call by Reference

```
#include <iostream>
void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
}
void applyConcession(long long& fee, int percent)
{
    fee -= fee * percent / 100;
}
int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Expected output:

```
after call by value : 45000
after call by reference: 40500
```

Call by value leaves the caller's fee unchanged because only the local copy is reduced. Call by reference directly modifies hostelFee.

1.5 Design Decision and Alternative

For a single fee update, changing the fee through a reference is convenient because the function clearly updates the supplied fee. If the function should calculate a value without mutating its input, returning the new fee is clearer:

```
long long applyConcession(long long fee, int percent)
{
    return fee - fee * percent / 100;
}
```

```
// Usage:
hostelFee = applyConcession(hostelFee, 10);
```

A const reference should be used when a function needs to read a large object without copying it and must not modify it. For a long long, copying is inexpensive; for a large Student object, passing by value may copy many fields, while reference and const reference avoid that object copy.

Parameter style	Time	Extra parameter-copy space	Can modify caller?
long long by value	O(1)	O(1) scalar copy	No
long long&	O(1)	O(1) reference/address-sized	Yes
const Student&	O(1) for this operation	O(1) reference/address-sized	No
Student by value	O(n) if n represents copied fields/contained data	Proportional to object size	Only local copy

1.6 Evidence to Add

[Insert screenshot of the three passing methods, predicted output, actual output and completed TODO program.]

Q2 – Return by Reference and Its Dangers (4 Marks)

2.1 Implementation

```
#include <iostream>
int& highestMark(int marks[], int n)
{
   int best = 0;
   for (int i = 1; i < n; ++i)
      if (marks[i] > marks[best]) best = i;
   return marks[best];
}
int& unsafeLocalReference()
{
   int value = 50;
   return value; // Dangling reference: value ceases to exist on return
}
int main()
{
   int marks[] = {78, 91, 84, 88};
   highestMark(marks, 4) += 2;
   std::cout << marks[1] << '\n';
}
```

Expected output for the safe part: 93. The function returns a reference to the actual array element containing the highest mark. The expression highestMark(marks, 4) += 2 is valid because the function returns a modifiable lvalue reference.

2.2 Why Returning a Local Reference Is Unsafe

A local variable has automatic storage duration and stops existing when its function returns. Returning a reference to it leaves the caller with a dangling reference. Compilers commonly warn that the function returns the address/reference of a local variable. Using that reference has undefined behaviour; the program may print an unexpected value, crash or appear to work by accident. The unsafe function should be shown only as a warning example and must not be dereferenced.

2.3 Reference, Const Reference and Copy

Return type	Caller can modify original?	Main property
int&	Yes	Aliases the original element; caller must ensure lifetime remains valid.
const int&	No through this reference	Avoids copying but still depends on the referred object remaining alive.
int	No	Returns an independent copy; safest when returning a local calculation.

2.4 Vector Growth and Alternatives

A std::vector may reallocate its storage when it grows beyond its capacity. Reallocation invalidates references, pointers and iterators to its elements. Therefore, a reference returned before growth must not be used after an operation that may reallocate the vector.

```
// Index approach
int highestIndex(const std::vector<int>& marks) {
    int best = 0;
    for (int i = 1; i < static_cast<int>(marks.size()); ++i)
        if (marks[i] > marks[best]) best = i;
    return best;
}
// Usage: marks[highestIndex(marks)] += 2;

// Pointer approach (valid only while the element address remains valid)
int* highestPointer(std::vector<int>& marks) {
    int best = 0;
    for (int i = 1; i < static_cast<int>(marks.size()); ++i)
        if (marks[i] > marks[best]) best = i;
    return &marks[best];
}
```

Approach	Find maximum	Extra space	Safety after vector growth
Reference	O(n)	O(1)	Invalidated if vector reallocates.
Index	O(n)	O(1)	Index can be used again after growth, but may identify a different logical element if elements are rearranged.
Pointer	O(n)	O(1)	Invalidated if vector reallocates.

The index is usually the most robust choice when the container may grow, because it can be applied to the vector again after growth. The index must still be checked against the current size.

2.5 Evidence to Add

[Insert safe reference output and compiler warning screenshot for the intentionally unsafe local-reference example.]

Q3 – Default Arguments and Function Overloading (6 Marks)

3.1 Cause of Ambiguity

The three original declarations overlap. A call such as charge(2.0) can match charge(double), charge(double, bool) using a default for the second parameter, and charge(double, bool, bool) using defaults for the last two parameters. The compiler cannot select one unique best overload, so the call is ambiguous.

3.2 Redesigned Functions

```
#include <iostream>
#include <string>
struct Parcel {
    double weightKg;
    bool express = false;
    bool insured = false;
};

double charge(int grams, bool express = false, bool insured = false) {
    double kg = grams / 1000.0;
    return kg * 50.0 + (express ? 40.0 : 0.0) + (insured ? 20.0 : 0.0);
}
```

```
double charge(double weightKg, bool express = false, bool insured = false) {
    return weightKg * 50.0 + (express ? 40.0 : 0.0) + (insured ? 20.0 : 0.0);
}
double charge(const Parcel& p) {
    return charge(p.weightKg, p.express, p.insured);
}
int main() {
    std::cout << charge(2) << '\n';
    std::cout << charge(2.0) << '\n';
    std::cout << charge(2.0f) << '\n';
    std::cout << charge(2000, true) << '\n';
    std::cout << charge(2.5, false, true) << '\n';
    std::cout << charge(Parcel{3.0, true, true}) << '\n';
}
```

Pricing rule used in this example: ₹50 per kilogram, plus ₹40 for express delivery and ₹20 for insurance. This is an illustrative rule because the assignment does not prescribe actual courier rates.

3.3 Predicting the Six Overload Selections

Call	Selected overload	Reason
charge(2)	charge(int, bool, bool)	Exact match to int; defaults supply false and false.
charge(2.0)	charge(double, bool, bool)	Exact match to double.
charge(2.0f)	charge(double, bool, bool)	float promotes to double; no float overload exists.
charge(2000, true)	charge(int, bool, bool)	Exact int and bool matches; insured uses default.
charge(2.5, false, true)	charge(double, bool, bool)	Exact match to double, bool, bool.
charge(Parcel{...})	charge(const Parcel&)	Argument is a Parcel object.

Overload resolution prefers an exact match over a promotion, and a promotion over a standard conversion when comparing viable candidates. Default arguments fill omitted trailing parameters; they do not create a different function overload. Defaults must be supplied from right to left because an argument cannot be omitted while a later parameter is explicitly supplied.

3.4 Alternative Design and Complexity

A single Parcel structure can hold all options. The caller constructs a Parcel and passes it to one function. This avoids a large family of overloads, but call sites may be more verbose. A function template is another option when the function genuinely needs to support different types generically.

Design	Runtime overload cost	Argument-copy space	Adding a new option
Overloads + defaults	No runtime overload-resolution cost; selection occurs at compile time	Parcel/reference choice determines copying	May require changing declarations and overloads.
Single Parcel parameter	No overload-resolution cost	O(1) reference if const Parcel&, object storage belongs to caller	Add a field to Parcel and update processing.
Function template	Compile-time instantiation/selection	Depends on parameter passing	Can support new compatible types with less duplication.

3.5 Evidence to Add

[Insert compiler output showing the original ambiguous call error and the redesigned program's six calls.]

Q4 – Specifying a Class with an Array Inside It (6 Marks)

4.1 SavingsAccount Class

```
#include <iostream>
#include <string>
class SavingsAccount {
private:
    long long accountNumber;
    double balance;
    double transactions[10];
    int count;

    void record(double amount) {
        if (count < 10) {
            transactions[count++] = amount;
        } else {
            for (int i = 1; i < 10; ++i)
                transactions[i - 1] = transactions[i];
            transactions[9] = amount;
        }
    }
public:
    SavingsAccount(long long number, double openingBalance)
        : accountNumber(number), balance(openingBalance), count(0) {}

    void deposit(double amount);
    void withdraw(double amount);
    void miniStatement() const;
};

void SavingsAccount::deposit(double amount) {
    if (amount <= 0) return;
    balance += amount;
    record(amount);
}
void SavingsAccount::withdraw(double amount) {
    if (amount <= 0 || amount > balance) return;
    balance -= amount;
    record(-amount);
}
void SavingsAccount::miniStatement() const {
    std::cout << "Account: " << accountNumber
              << " Balance: " << balance << '\n';
    std::cout << "Recent transactions:\n";
    for (int i = 0; i < count; ++i)
        std::cout << transactions[i] << '\n';
}
```

The transaction array, count, account number and balance are private. This protects the account state from direct, uncontrolled modification. Public functions provide controlled operations and can enforce rules such as rejecting negative deposits and withdrawals greater than the balance.

4.2 Handling the 11th Transaction

Design	When full	Insertion cost	Space
Shift array	Move entries 1–9 one position left, then write new transaction at index 9.	O(k), where k is the number of stored entries; here at most 10.	O(10), fixed.
Circular buffer	Overwrite the oldest slot and advance the start index modulo capacity.	O(1).	O(10), fixed.
std::deque	push_back(new); pop_front() when size exceeds 10.	Typically O(1) at the ends.	O(10) logical retained entries, with container overhead.

For 50 transactions, the capacity constant and array size would need to change from 10 to 50. A circular buffer or deque is easier to scale without shifting every retained transaction. For a small fixed capacity, a fixed array is simple and predictable.

4.3 Alternative Circular-Buffer Pseudocode

```
capacity = 10
transactions[capacity]
start = 0
count = 0

record(value):
    if count < capacity:
        index = (start + count) % capacity
        transactions[index] = value
        count = count + 1
    else:
        transactions[start] = value
        start = (start + 1) % capacity
```

Adding a transaction to the circular buffer takes O(1) time and O(10) storage. A shifting array takes O(k) time for each insertion after it becomes full.

4.4 Evidence to Add

[Insert source file and output screenshot showing deposits, withdrawals, balance and the last 10 transactions after more than 10 operations.]

Q5 – Static Members and Arrays of Objects (7 Marks)

5.1 Static Member Design

The number of issued seats belongs to the BusTicket class as a whole, not to an individual ticket. Therefore, the issued counter must be static. The seat number and passenger name are per-object data and must remain non-static.

```
#include <iostream>
#include <string>
class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n) {
        if (issued < CAPACITY) {
            ++issued;
            seat = issued;
        }
    }
    static int count() {
        return issued;
    }
    void print() const {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat)
                           : std::string("Waiting")) << '\n';
    }
};
int BusTicket::issued = 0;

int main() {
    BusTicket trip[4] = {
        BusTicket("Asha"), BusTicket("Bala"),
        BusTicket("Chitra"), BusTicket("Dev")
    };
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

Expected output:

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

5.2 Static Member Function

A static member function belongs to the class rather than a particular object. It has no this pointer, because it is not called for one specific instance. It can directly access static members, but it cannot directly access non-static members because those require a particular object's identity.

The static data member is declared inside the class and defined once outside the class: int BusTicket::issued = 0;. The array trip[4] constructs four objects. The first three receive seats and the fourth is placed on the waiting list.

5.3 Constructor Calls in the Token Program

```
Token counter[3];
Token extra;
std::cout << counter[0].get() << ' '
        << counter[2].get() << ' '
        << extra.get() << ' '
        << Token::issued() << '\n';
```

The static next variable starts at 100. The three array elements and the extra object each run the constructor once, incrementing next. Their IDs are 101, 102, 103 and 104. issued() returns next - 100, which is 4.

Predicted output: 101 103 104 4

5.4 Creating Tickets in a Function Called Twice

```
void createTemporaryTickets() {
   BusTicket local1("Temporary 1");
   BusTicket local2("Temporary 2");
}
```

The static issued count belongs to the class and survives after local objects are destroyed. Calling this function twice continues incrementing the same count; destroying the local tickets does not reduce the issued counter. If the counter is intended to reset for a new bus trip, the design should represent each bus trip with a separate TicketOffice object or a separate trip-specific counter rather than one global class-wide count.

5.5 Alternative and Complexity

Design	Time to issue	Space	Encapsulation / multiple buses
Static class member	O(1)	One shared counter	Encapsulated in class, but all buses share one count.
Global variable	O(1)	One global counter	Weak encapsulation; accidental access/modification is easier.
TicketOffice object passed to ticket	O(1)	One counter per office	Supports multiple buses independently and makes ownership explicit.

For a system that may operate two buses at once, a TicketOffice object per bus is the strongest design because each office maintains its own capacity and issued count.

5.6 Evidence to Add

[Insert output screenshot for the Token program and BusTicket program, including the four-ticket waiting-list case and function-called-twice test.]

Q6 – Friend Functions and Inline Functions (8 Marks)

6.1 Completed Friend Functions

```
#include <iostream>
#include <cstdlib>
class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};
int minutesBetween(const Time& a, const Time& b) {
    int first = a.h * 60 + a.m;
    int second = b.h * 60 + b.m;
    return std::abs(first - second);
}
bool earlier(const Time& a, const Time& b) {
    if (a.h != b.h) return a.h < b.h;
    return a.m < b.m;
}
int main() {
    Time departure(9, 45), arrival(13, 10);
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha << earlier(arrival, departure) << '\n';
}
```

Expected output: 205 false. 09:45 is 585 minutes after midnight and 13:10 is 790 minutes after midnight. The absolute difference is 205 minutes. Arrival is not earlier than departure.

6.2 Additional Correctness Tests

```
Time morning(11, 50), afternoon(12, 10), sameA(8, 30), sameB(8, 30);
// minutesBetween(morning, afternoon) -> 20
// earlier(morning, afternoon) -> true
// minutesBetween(sameA, sameB) -> 0
// earlier(sameA, sameB) -> false
```

The first test crosses noon and should report 20 minutes, with the morning time earlier. The second test uses equal times and should report zero minutes and false for strictly earlier.

6.3 Friend Function vs Member Function

Feature	Friend function	Member function
Call syntax	minutesBetween(a, b)	a.minutesTo(b)
Object relationship	Treats both objects symmetrically	One object is the receiver (this).
Private access	Can access private data because it is declared friend	Can access private data of its own object and permitted objects.
Encapsulation	Grants a specific external function access	Keeps operation within class interface.

A friend function is suitable when an operation naturally treats two Time objects equally. It is not a member function even though it has access to private members. Making h and m public would weaken encapsulation because any code could assign invalid values such as hour 29 or minute 90.

6.4 Inline Functions and Timing Experiment

A function defined inside a class definition is implicitly inline. The inline keyword is a request/permission for the compiler to consider inline substitution; modern compilers may inline or not inline based on optimisation and their own analysis. Inline does not guarantee that a function body is physically substituted at every call.

The assignment asks for 10,000,000 calls with and without inline at -O0 and -O2. Use the same workload and compiler for a fair comparison. Avoid printing inside the timed loop because I/O would dominate the measurement.

```
// Suggested timing experiment structure
#include <chrono>
```

```
#include <iostream>

inline int addInline(int a, int b) { return a + b; }
int addNormal(int a, int b) { return a + b; }

int main() {
    volatile long long total = 0;
    const int repetitions = 10000000;

    auto start = std::chrono::steady_clock::now();
    for (int i = 0; i < repetitions; ++i)
        total += addInline(i, 2);
    auto middle = std::chrono::steady_clock::now();

    for (int i = 0; i < repetitions; ++i)
        total += addNormal(i, 2);
    auto finish = std::chrono::steady_clock::now();

    std::cout << "Inline loop (ns): "
              << std::chrono::duration_cast<std::chrono::nanoseconds>(middle-start).count() << '\n';
    std::cout << "Normal loop (ns): "
              << std::chrono::duration_cast<std::chrono::nanoseconds>(finish-middle).count() << '\n';
    std::cout << "Check: " << total << '\n';
}
```

Compile and run separately with:

```
g++ -std=c++17 -O0 timing.cpp -o timing_O0
./timing_O0

g++ -std=c++17 -O2 timing.cpp -o timing_O2
./timing_O2
```

NOTE: The timing code is a starting point. The volatile accumulator reduces the chance that the compiler removes the work, but benchmark results still depend on CPU, compiler, system load and optimisation. Run several trials and report the median. Do not copy invented timing values into the report.

Optimisation	Inline function time	Non-inline function time	Trial / median notes
-O0	[Enter measured result]	[Enter measured result]	[Enter 3 or more trials]
-O2	[Enter measured result]	[Enter measured result]	[Enter 3 or more trials]

6.5 Deployment Decision

The friend functions can remain if the comparison is naturally symmetric and the class should keep its data private. Member functions with getters are an alternative when the operation belongs conceptually to one Time object or when friend access should be minimised. Getters preserve private storage but expose read access through the public interface. All three approaches can perform O(1) work per comparison; measured differences depend on compiler optimisation.

6.6 Evidence to Add

[Insert correctness-test output, -O0 and -O2 timing screenshots, compiler command and compiler version.]

Q7 – Diagnose, Redesign and Validate (13 Marks)

7.1 Selected Limitation: Savings Account Transaction History

This report selects the fixed-array transaction history in Q4 as the redesign target. When the array is full, the original shift-based design moves the remaining nine entries for every new transaction. The work increases with the number of retained transactions k, giving O(k) insertion work after the history is full.

7.2 Measurable Design Objective

Objective: retain only the latest 10 transactions while reducing the work required to record a new transaction from O(k) shifting to O(1) indexed insertion. The redesigned implementation must produce the same ordered mini-statement as the original for the same sequence of deposits and withdrawals.

7.3 Revised Solution: Circular Buffer

```
class TransactionHistory {
private:
    static const int CAPACITY = 10;
    double values[CAPACITY]{};
    int start = 0; // index of the oldest stored item
    int count = 0;
public:
    void add(double value) {
        if (count < CAPACITY) {
            int index = (start + count) % CAPACITY;
            values[index] = value;
            ++count;
        } else {
            values[start] = value;
            start = (start + 1) % CAPACITY;
        }
    }
    void print() const {
        for (int i = 0; i < count; ++i) {
            int index = (start + i) % CAPACITY;
            std::cout << values[index] << '\n';
        }
    }
};
```

The circular buffer stores a fixed array, the index of the oldest item and the number of valid entries. While not full, the next value is written after the newest entry. Once full, the oldest slot is overwritten and start advances.

7.4 Class Design / Pseudocode

```
TransactionHistory
  - values[10]
  - start
  - count
  + add(transaction)
  + print()

add(transaction):
  if count < 10:
      write at (start + count) modulo 10
      increment count
  else:
      overwrite values[start]
      advance start modulo 10

print():
  for each logical position from oldest to newest:
      print values[(start + position) modulo 10]
```

7.5 Prediction Before Implementing

Prediction: both implementations will retain and display the same latest 10 transactions. The shifting design will perform up to nine element moves per insertion after becoming full. The circular-buffer design will perform one indexed write and one index update, so its insertion work should remain constant as the buffer stays full. Both use fixed O(10) storage.

7.6 Validation Plan and Results

Run both versions with the same input sequence containing fewer than 10 transactions, exactly 10 transactions and at least 15 transactions. Compare the retained values and their order. Measure insertion time using a large number of additions, repeat each measurement and record the median.

Test case	Input	Expected retained	Original result	Redesigned result

		history		
Below capacity	6 transactions	All 6 in original order	[Enter]	[Enter]
At capacity	10 transactions	All 10 in original order	[Enter]	[Enter]
Above capacity	15 transactions	Transactions 6 through 15	[Enter]	[Enter]

Measurement	Shift-based array	Circular buffer
Insertion complexity after full	O(k)	O(1)
Storage	O(10)	O(10)
Measured time	[Enter actual measurement]	[Enter actual measurement]
Output correctness	[Pass/Fail]	[Pass/Fail]

7.7 Final Evaluation

The redesign meets the objective if it preserves the latest 10 transactions in the correct order and reduces insertion work to O(1). Its trade-off is additional index logic, which makes printing and debugging slightly less straightforward. The original shifting array may remain preferable when the capacity is very small and implementation simplicity matters more than insertion efficiency. The final conclusion must be based on the student's own output comparison and measured results.

7.8 Evidence to Add

[Insert both source files, matching test input/output screenshots, timing results and a short comparison.]

Overall Complexity Summary

Question	Core operation	Time complexity	Auxiliary space
Q1	Apply one percentage concession	O(1)	O(1)
Q2	Find highest mark in n marks	O(n)	O(1)
Q3	Calculate charge after overload selection	O(1) calculation; overload resolution at compile time	O(1) for scalar arguments; object copy depends on signature
Q4	Record transaction in shift array / circular buffer	O(k) / O(1)	O(10)
Q5	Issue one ticket	O(1)	O(1) shared counter, plus objects
Q6	Compare two times	O(1)	O(1)
Q7	Record transaction in redesigned circular buffer	O(1)	O(10)

Conclusion

This assignment demonstrates how C++ parameter passing, references, pointers, default arguments, overloading and object-oriented features affect correctness, safety, maintainability and performance. The evaluation also shows that a design should be selected according to its requirements: references are useful for controlled modification, const references avoid unnecessary object copies, private members protect class invariants, static members represent class-wide state, and a circular buffer provides constant-time insertion for a fixed-size recent-history requirement.

Final Verification Checklist

- All Q1–Q6 programs compile and execute successfully.
- Predictions are written before execution and compared with actual output.
- All TODO sections are completed without unnecessary changes to supplied code.
- Compiler warnings are captured where requested, especially for the dangling reference demonstration.
- Q6 timings are measured on the user's computer at both -O0 and -O2, with repeated trials.
- Q7 redesign is tested against the original using identical input.
- All placeholders for actual outputs, screenshots, compiler details and measurements are replaced.
- GitHub repository contains task folders, source code, screenshots, README and progressive commits.
- GitHub link is present on the cover page.

Appendix A – Suggested GitHub Folder Structure

```
DSA01-<RegisterNumber>-Assignment2/
|-- README.md
|-- Task01/
|   |-- q1_prediction.cpp
|   |-- q1_concession.cpp
|   `-- output_screenshots/
|-- Task02/
|-- Task03/
|-- Task04/
|-- Task05/
|-- Task06/
|   |-- q6_friend_functions.cpp
|   `-- timing.cpp
|-- Task07/
|   |-- original_shift_buffer.cpp
|   `-- redesigned_circular_buffer.cpp
`-- report/
    `-- Assignment2_CO2_Report.docx
```

Appendix B – Suggested README Build Command

```
g++ -std=c++17 -Wall -Wextra -pedantic filename.cpp -o program
./program
```

On Windows Command Prompt, run the generated executable as program.exe. Record the exact compiler version using g++ --version and include it in README.md.

Detailed Discussion and Evaluation – Q1

1.7 Detailed Working of Parameter Passing

A function receives information through its parameters. The way a parameter is declared determines whether the function works with a separate value, an alias to the original object, or an address that can be dereferenced. These are not merely different syntaxes; they represent different contracts between the caller and the function.

In call by value, the argument expression is used to initialise a new parameter object. Any assignment to that parameter changes only the parameter object. This behaviour is useful when a function should work independently of the caller's variable. It also prevents accidental changes to the caller's state. The disadvantage is that copying a large object can consume time and memory.

In call by reference, the parameter is another name for the argument supplied by the caller. The function can therefore update the original variable without returning it. A reference must be initialised when declared and normally cannot be reseated to refer to another object. This makes references convenient for functions that require a valid object.

In pointer passing, the function receives an address. The pointer itself is passed by value, but it can point to the caller's object. Dereferencing the pointer accesses that object. A pointer can also be null, so a robust function should define what happens when the supplied pointer is null.

1.8 Concession Function Design

A concession function should use integer arithmetic consistently with the expected fee type. The expression fee * percent / 100 calculates the discount amount, and subtracting that amount produces the revised fee. Integer arithmetic discards any fractional remainder. If the business rule requires paise or fractional currency, the representation and rounding rule must be specified rather than relying on an accidental conversion.

When several concessions may apply, the order and combination rule must be clarified. For example, applying 10% and then 15% sequentially gives a different result from adding the percentages and applying 25% once. The assignment asks the student to defend whether the function modifies the fee in place or returns a new fee. A reference-based update is appropriate when mutation is the intended operation. A return value is preferable when the calculation should be explicit, testable and easy to compose.

1.9 Test Cases and Interpretation

Test	Starting fee	Percent	Expected result	Purpose
Normal concession	45000	10	40500	Confirms percentage calculation.
No concession	45000	0	45000	Checks zero-percent boundary.
Full concession	45000	100	0	Checks upper meaningful boundary.
Small fee	99	10	90	Shows integer truncation.
Multiple updates	45000	10 then 15	34425	Illustrates sequential application.

The final row is an illustration of sequential discounts: after a 10% reduction, the fee becomes 40,500; a further 15% reduction gives 34,425. The report should state the policy used if more than one concession can apply.

For the viva, be prepared to explain why applyByValue() prints no updated caller value, why applyConcession(long long&) does, and why a pointer-based implementation must dereference the pointer. Also explain when const long long& or const Student& is suitable.

Detailed Discussion and Evaluation – Q2

2.6 Lifetime and Object Validity

The central rule in return-by-reference design is that the referred object must continue to exist for as long as the caller uses the reference. An array declared in main() remains alive until main() ends, so returning a reference to one of its elements is valid while the array remains in scope. A local variable declared inside a function is destroyed when the function returns, so a reference to it immediately becomes dangling.

A dangling reference is dangerous because the language does not guarantee what happens when the program accesses the invalid object. The memory location may later be reused for another value. A program that appears to work on one run may fail after a compiler change, optimisation change or unrelated code modification. Therefore, undefined behaviour must not be treated as a predictable output.

2.7 Safe Function Contracts

A function returning int& promises that the caller may modify the referred integer. A function returning const int& promises read-only access through that returned reference. A function returning int provides a separate value and does not expose the original storage. The choice should be visible in the function's name, return type and documentation.

For a growing vector, the student must consider invalidation. A vector stores elements contiguously. When capacity is insufficient, it may allocate a new memory block and move or copy elements into it. References and pointers to the old block then become invalid. Calling reserve() may reduce the chance of reallocation for a known workload, but it is not a general guarantee if the vector later exceeds its reserved capacity.

2.8 Example of Reference Invalidation

Suppose a vector contains marks and a function returns a reference to the maximum element. The caller stores that reference. If the caller then appends enough marks to force reallocation, the stored reference must no longer be used. A safer workflow is to store the maximum element's index, allow the vector to grow, verify that the index is still within the vector's size, and then access the element again.

Return strategy	Advantages	Risks / limitations
Reference	Direct modification; convenient expression syntax.	Depends on object lifetime and storage stability.
Const reference	Avoids copying and prevents modification through this access path.	Still dangles if the object is destroyed or invalidated.
Value copy	Independent result; simple lifetime rules.	Does not update the original element.
Index	Can be reapplied after vector reallocation.	May refer to a different logical item if elements are reordered or erased.
Pointer	Can express optional absence using nullptr.	Lifetime and invalidation risks remain.

2.9 Validation Checklist

- Confirm that the highest-mark function returns the correct element for an unsorted array.
- Test an array containing one element.
- Test repeated maximum values and document which occurrence the implementation returns.
- Modify the returned reference and print the original array to prove that the array element changed.
- Compile the unsafe local-reference example separately and capture the compiler diagnostic.
- Do not execute a dereference of the dangling reference as if its output were reliable.

Detailed Discussion and Evaluation – Q3

3.6 Understanding Overload Resolution

Function overloading allows multiple functions to share a name when their parameter lists differ. The compiler examines the argument types and identifies viable candidates. It then ranks the conversions required by each candidate. An exact match is preferred to a promotion, and a promotion is generally preferred to a standard conversion. If two candidates remain equally good, the call is ambiguous and the program is rejected.

Default arguments do not form part of the function's signature. They simply provide values for omitted trailing parameters after a function has been selected. This distinction is important: adding defaults to several overloads can create overlapping viable candidates even though the declarations look different.

3.7 Why the Redesigned Interface Is Clearer

The redesigned example uses an int overload for grams, a double overload for kilograms and a Parcel overload for a structured parcel. This makes the intended unit visible in the type used at the call site. The example uses an illustrative pricing formula so that overload selection can be demonstrated; it is not a rate supplied by the assignment question.

A production system should consider stronger unit types because a plain integer does not communicate whether the number means grams, kilograms, pieces or another unit. A named structure or distinct type can reduce unit mistakes. The Parcel structure also groups optional service choices together and can be extended with tracking, fragile handling or delivery-zone information.

3.8 Additional Test Matrix

Call	Expected selected function	Key language rule
charge(2)	int overload	Exact int match.
charge(2.0)	double overload	Exact double match.
charge(2.0f)	double overload	float-to-double promotion.
charge(2000, true)	int overload	Exact first and second parameter matches; trailing default.
charge(2.5, false, true)	double overload	All supplied arguments match.
charge(Parcel{...})	Parcel overload	Exact class-type match.

3.9 Design Trade-Off

Overloads improve readability when each input type represents a meaningful variation. Too many overloads, however, can make calls difficult to reason about. A single structure makes optional settings explicit and scales better when the number of options grows. A template is useful when the implementation is genuinely type-independent, but it should not be introduced only to avoid writing a small number of clear overloads.

The report should include the compiler diagnostic from the original ambiguous design, followed by the successful compile and run of the redesigned version. The student should be able to explain why charge(2) and charge(2.0f) choose different candidates.

Detailed Discussion and Evaluation – Q4

4.5 Class Encapsulation and Invariants

A class combines data and the operations that are permitted on that data. In SavingsAccount, the balance and transaction history should not be directly accessible to unrelated code. If they were public, a caller could assign an invalid balance or change the transaction count without recording a transaction. Keeping data private ensures that changes pass through the class's public operations.

An invariant is a condition that should remain true throughout the object's lifetime. Examples for this account include a non-negative balance, a transaction count between zero and ten, and a mini-statement containing no more than ten recent transactions. deposit() and withdraw() are responsible for preserving these conditions.

4.6 Inside-Class and Outside-Class Definitions

The constructor and a small helper can be defined inside the class body. The deposit(), withdraw() and miniStatement() functions can be declared inside the class and defined outside using the scope resolution operator. The syntax SavingsAccount::deposit identifies the function as a member of SavingsAccount. This separation makes the class interface easier to read while keeping longer implementation details outside the declaration.

4.7 Shift Array Versus Circular Index

The shift-array method is straightforward. When a new transaction arrives after the array is full, each existing item is moved one position toward the beginning and the new item is stored at the end. The method preserves chronological order in memory, which makes printing simple. Its cost is proportional to the number of stored items.

The circular-buffer method avoids moving existing values. It remembers the oldest slot with a start index. When the buffer is full, the next transaction overwrites the oldest slot and the start index advances. The data is physically wrapped around the array, so printing must calculate each logical index using modulo arithmetic. This introduces a little more logic but keeps insertion constant time.

Requirement	Shift array	Circular buffer
Simple implementation	Strong	Moderate
Insertion after full	O(k)	O(1)
Printing chronological order	Direct loop	Loop with modulo index
Fixed memory use	Yes	Yes
Suitable for larger fixed capacity	Works, but more movement	Efficient

4.8 Boundary and Error Tests

- Deposit a positive amount and confirm that balance increases.
- Withdraw an amount smaller than the balance and confirm that balance decreases.
- Attempt to withdraw more than the available balance and confirm that the operation is rejected.
- Attempt a zero or negative deposit and confirm that the class does not record an invalid transaction.
- Perform at least 12 transactions and verify that only the most recent 10 remain.
- Check that the mini-statement prints transactions from oldest retained to newest retained.

Detailed Discussion and Evaluation – Q5

5.7 Meaning of Class-Wide State

An ordinary data member belongs to one object. If ten BusTicket objects are created, each object has its own seat and passenger name. A static data member belongs to the class itself and is shared by every object of that class. It is therefore suitable for a class-wide issued-seat counter when all ticket objects represent the same bus capacity.

The static member is declared in the class and, in the traditional C++ form used by the assignment, defined once outside the class. This definition provides storage for the shared member. The static count() function can be called through the class name because it does not need a particular ticket object.

5.8 Object Construction and Waiting List

The BusTicket constructor initialises the seat to zero and stores the passenger name. It then checks whether the shared issued count is below capacity. If a seat is available, it increments the count and assigns the new count as the seat number. If capacity has already been reached, seat remains zero and print() displays Waiting.

An array of objects constructs each element. The four-element trip array in the assignment therefore invokes the constructor four times. Only three objects receive a seat because the capacity is three. The fourth object is still a valid BusTicket object, but its seat field indicates that it is waiting.

5.9 Static Counter Versus Ticket Office Object

A single static counter is simple when the application has exactly one bus or one shared capacity pool. It becomes limiting when two buses operate independently. If all tickets share one static issued counter, passengers for the second bus may incorrectly consume the first bus's capacity.

A TicketOffice object provides a separate counter and capacity for each bus. A ticket can be associated with an office, and two offices can issue tickets independently. This design also improves testability because a test can create a fresh office with a known capacity. The global-variable alternative has similar shared-state problems and weaker encapsulation.

Scenario	Static counter	Separate TicketOffice objects
One bus	Simple and suitable	Works, but may be more structure than needed.
Two independent buses	Shared counter causes incorrect coupling unless redesigned.	Each bus has its own count.
Reset a new trip	Requires carefully controlled reset logic.	Create a new office/trip object.
Testing	Shared state can leak between tests.	Each test can create a fresh office.

5.10 Additional Tests

- Create exactly three tickets and verify seats 1, 2 and 3.
- Create a fourth ticket and verify that it is Waiting.
- Create the array in a function and observe that the static count persists after the function returns.
- Call the ticket-creation function twice and explain why the count continues rather than resetting.
- Test separate office objects if the design is extended to support two buses.

Detailed Discussion and Evaluation – Q6

6.7 How Friend Functions Work

A friend function is an ordinary non-member function that is granted access to a class's private and protected members. The class declares the function with the friend keyword. The function is defined outside the class and called like a normal function, not through an object using the member-function dot syntax.

The minutesBetween(a, b) operation treats both Time objects equally. It converts each time into minutes after midnight and returns the absolute difference. The earlier(a, b) operation compares hours first and minutes second. These functions need access to the private h and m members, so the class grants them friendship.

Friendship should be granted only when it improves the design. Making all data public would allow any part of the program to create invalid times. A friend declaration is narrower: it grants access to the named function while the data remains private to other code.

6.8 Inline Does Not Guarantee Faster Execution

The inline keyword historically suggested that a function body could be substituted at the call site. In modern C++, it also has language and linkage implications, especially for definitions in headers. The compiler is free to make its own optimisation decision. A function marked inline may still be called normally, and a function not marked inline may be inlined by an optimiser.

At -O0, the compiler generally performs fewer optimisations, so call overhead may be more visible. At -O2, the compiler may inline small functions, remove redundant work and optimise loops. Therefore, timing differences between the two versions may become very small or may be affected by other optimisations.

6.9 Fair Measurement Method

1. Use the same source logic and compiler for both builds.
2. Compile one executable with -O0 and another with -O2.
3. Run each executable several times under similar system conditions.
4. Do not include console output inside the measured loop.
5. Use a result accumulator so the computation has an observable effect.
6. Record compiler version, CPU/system conditions, each trial and the median.
7. Explain that microbenchmark results are specific to the machine and compiler.

6.10 Interpretation

The theoretical time complexity of comparing two times is O(1), because it performs a fixed number of arithmetic operations and comparisons. The auxiliary space is O(1). Inline and non-inline versions do not change the asymptotic complexity. The measured execution time may differ, but the conclusion must be based on actual repeated measurements rather than an assumed result.

Test input	minutesBetween	earlier result
09:45 and 13:10	205	earlier(09:45, 13:10) = true
11:50 and 12:10	20	earlier(11:50, 12:10) = true
08:30 and 08:30	0	earlier(08:30, 08:30) = false
13:10 and 09:45	205	earlier(13:10, 09:45) = false

Expanded Design Report – Q7

7.9 Why a Redesign Is Necessary

The purpose of Q7 is not simply to replace an array with a library container. It asks the student to identify a real limitation in a previous task, define a measurable objective, design a solution and validate it using comparable evidence. The selected transaction-history example is suitable because the original implementation has a visible repeated operation: shifting all retained values to make room for a new value.

The original method is easy to understand and has predictable memory use. However, after the history reaches capacity, every new transaction requires several assignments. If the retained history size is k, the insertion cost is O(k). For a capacity of ten, this may not matter in practice, but the assignment's changed-condition discussion asks what happens when the required history grows to 50 or more entries.

7.10 Requirements for the Revised Design

Requirement ID	Requirement	Verification method
R1	Retain no more than 10 transactions.	Add more than 10 and count printed entries.
R2	Retain the newest transactions.	Compare output against the last 10 input values.
R3	Print in chronological order.	Check oldest retained value appears first.
R4	Add each transaction in constant-time indexed work.	Inspect implementation and benchmark repeated insertions.
R5	Use fixed-size storage.	Confirm storage array remains capacity 10.

7.11 Detailed Circular Buffer Behaviour

Assume a capacity of five for a simple illustration. The first five values are stored at indices 0, 1, 2, 3 and 4. The start index is zero and count is five. When a sixth value arrives, it replaces the value at index zero, because that is the oldest entry. The start index advances to one. The logical order is now the values at indices 1, 2, 3, 4 and 0. The physical order and logical order are different, but the modulo calculation preserves the correct chronological display.

The same method works with capacity ten. The start index always identifies the oldest retained transaction. The count indicates how many positions contain valid entries. Before the buffer is full, count increases. Once it is full, count stays at capacity and start advances after each overwrite.

7.12 Trade-Offs and Risks

- The circular buffer has slightly more complicated indexing than the shift-based implementation.
- Modulo arithmetic must use the correct capacity and handle wrap-around.
- The output loop must begin at start rather than assuming the oldest value is at index zero.
- The buffer is fixed-size; changing capacity still requires changing its storage design unless capacity is made configurable.
- The implementation uses no dynamic allocation for the stored transactions.
- The shift-based design may be easier to maintain for very small histories where insertion performance is not important.

Q7 – Validation, Comparison and Final Defence

7.13 Controlled Experiment

A valid comparison must use identical transaction sequences and the same computer. If one implementation receives 100,000 transactions and the other receives 10,000,000, their timings cannot be compared fairly. The test should first establish correctness and then measure performance.

8. Prepare a deterministic sequence of transaction values so both versions receive exactly the same data.
9. Run a correctness test with 6 values, then 10 values, then 15 values.
10. Verify that the 15-value case retains values 6 through 15 in the correct order.
11. Run a larger insertion workload for timing, such as repeated batches of additions.
12. Measure several trials for each version and calculate the median.
13. Record the compiler, compiler flags, machine and workload size.
14. Compare both output correctness and measured performance.

7.14 Results Table – Complete After Execution

Workload	Shift implementation	Circular buffer	Interpretation
6 insertions	[Actual result]	[Actual result]	Both should retain all six values.
10 insertions	[Actual result]	[Actual result]	Both should retain all ten values.
15 insertions	[Actual result]	[Actual result]	Both should retain values 6–15.
Large workload timing	[Measured time]	[Measured time]	Compare repeated-trial median.

7.15 Evaluation Against Objective

The first part of the objective is functional: both versions must retain exactly the latest ten transactions and display them in chronological order. The second part is algorithmic: the circular buffer should perform a constant number of indexed operations per insertion, while the shifting implementation moves up to nine existing entries once full. The third part is empirical: the measured benchmark should be consistent with the expected reduction in work, while recognising that small workloads and system noise can obscure the difference.

A redesign is not automatically successful because its Big-O complexity is better. It must also preserve the required behaviour, remain understandable and avoid introducing correctness defects. If the measured times are similar for a tiny capacity, the report should explain that the constant factors and small k make the difference difficult to observe.

7.16 Final Defence

The circular buffer is recommended when the system continuously records recent events and insertion efficiency is important. The shift-based array remains acceptable when the capacity is very small, the number of insertions is low and implementation simplicity is the priority. The final decision should be supported by the correctness tests and timing measurements recorded by the student.

7.17 Evidence Checklist for Q7

- Original shift-based source code.
- Redesigned circular-buffer source code.
- Same test data applied to both programs.
- Output screenshots for below-capacity, full-capacity and overflow cases.
- Timing measurements with compiler and system details.
- Explanation of any difference between predicted and observed results.
- Final conclusion stating whether the objective was achieved.

Implementation, Compilation and GitHub Documentation

8.1 Recommended Development Workflow

Each question should be developed and tested in its own folder. Keeping the programs separate makes it easier to identify compilation errors, capture output screenshots and demonstrate progress during the viva. The source file should have a meaningful name that identifies the concept being tested.

15. Create the repository with the exact name required by the assignment.
16. Create Task01, Task02, Task03, Task04, Task05, Task06 and Task07 folders.
17. Write the source file for the task and compile it with warnings enabled.
18. Resolve warnings and errors that are relevant to the program.
19. Run the program and compare its output with the prediction or expected output.
20. Save a screenshot that clearly shows the command and result.
21. Update the README with the purpose, compile command and result.
22. Commit the task before starting the next task.

8.2 Compilation Commands

The assignment gives g++ as an example compiler and asks students to record the exact command and compiler version used. A useful command for standard C++17 compilation with common warnings enabled is:

```
g++ -std=c++17 -Wall -Wextra -pedantic source.cpp -o program
```

On Windows Command Prompt, run the executable as program.exe. On Linux or macOS, run it as ./program. Record the output of g++ --version in the README so the execution environment is reproducible.

8.3 Suggested README Content

The README should identify the student, register number, compiler and version, repository purpose and folder structure. For each task, include a short explanation, the exact compile command, the run command and a brief result statement. The README should not claim that a program passes unless it has actually been compiled and run.

8.4 Screenshot Quality

- Capture the complete terminal or IDE output, not a cropped fragment that hides the command.
- Use readable font size and ensure the program name is visible.
- Keep each screenshot inside the corresponding task folder.
- Use clear names such as Q1_call_by_reference_output.png.
- Do not edit output text to make a failed test look successful.
- Include compiler warnings as evidence where the assignment specifically asks for them.

8.5 Progressive Commit Examples

```
git add Task01 README.md
git commit -m "Complete Q1 parameter passing"
git add Task02
git commit -m "Complete Q2 return by reference"
git push
```

Use meaningful commit messages that describe the work completed. The assignment explicitly states that commit history will be checked, so a single final upload of every file is not an acceptable substitute for progressive work.

Viva Preparation and Concept Review

9.1 Important Questions to Practise

What is the difference between call by value and call by reference?

Call by value creates a separate parameter copy. Call by reference gives the function an alias to the caller's object, so changes affect the caller.

Why does a pointer version modify the original value?

The pointer contains the address of the original object, and dereferencing it accesses that object.

What is a dangling reference?

A reference that refers to an object whose lifetime has ended or whose storage has been invalidated.

Why is returning a reference to a local variable unsafe?

The local object is destroyed when the function returns, leaving the returned reference invalid.

What is a default argument?

A value supplied by the compiler for an omitted trailing parameter when the function is called.

Why can overloaded functions become ambiguous?

More than one viable overload may require equally good conversions for the supplied arguments.

Why are class data members private?

To protect the object's state and ensure changes pass through controlled member functions.

What is a static data member?

A class-wide member shared by all objects of that class.

Why can a static member function not use this?

It is not called for one particular object and therefore has no this pointer.

What is a friend function?

A non-member function granted access to a class's private or protected members.

Does inline guarantee that a function call is removed?

No. It is not a guarantee of machine-code substitution; the compiler decides optimisation.

What is the advantage of a circular buffer?

It overwrites the oldest entry using constant-time indexed operations instead of shifting retained entries.

9.2 Final Academic Integrity Reminder

This report provides explanations, sample implementations and a structure for presenting the work. The student must compile and understand every program, write predictions before running the relevant programs, capture genuine evidence and replace all bracketed placeholders with personal details and actual results. The report should be submitted only after the GitHub repository and execution evidence are complete.